Trade-Off Template (TCD Section 5)

> **Day 5 · Activity 2 handout.** TCD Section 5 is the section that separates senior architectural thinking from feature-list thinking. A trade-off is not a list of risks — it's a structured statement: *we considered A and B, we chose A, we accept this cost, this is what would change our minds.*

The pattern

Option A vs Option B → choice → accepted cost → revisit trigger.

Immature	Mature
"We considered microservices but chose a monolith"	"Option A: modular monolith. Option B: extract reconcile into a service. We chose A. We accept that future service-extraction will require coordinated refactoring; we will revisit if traffic exceeds 5x rest-of-app or if a second team starts contributing reconcile features."
"Performance vs observability is a trade-off"	"Logging full event detail adds ~30ms to the hot path; we accept that until p95 nears the SLO; if the budget closes, we move to async event publication."

The five categories

A well-written Section 5 spans **at least 3** of these. Five trade-offs all in one category usually means real tensions were missed.

Category	Example tension
Coupling vs independence	Module in monolith vs new service
Consistency vs availability	Strong-consistency reads vs replica reads
Latency vs durability	Sync write vs async with retry
Speed vs robustness	Ship the happy path now, harden later
Generality vs simplicity	Build the generic capability or just this case

---

The template (one per trade-off, five total)

```
### Trade-off N — <short name>
**Tension:** <category> — <one sentence framing the tension>
**Option A:** <description, 1–2 sentences>
**Option B:** <description, 1–2 sentences>
**Choice:** Option A.
**Why:** <2–3 sentences — defended in business + operational terms>
**Accepted cost:** <what we give up, concretely>
**Revisit trigger:** <metric or organizational change>
```

Worked examples — FieldPulse reconcile

```
### Trade-off 1 — Module vs new service
**Tension:** Coupling vs independence.
**Option A:** Reconcile lives as a module inside the existing backend
monolith.
**Option B:** Extract reconcile into a new microservice with its own
database.
**Choice:** Option A.
**Why:** Only one team owns reconcile; deploy-cadence contention is
zero.
        Reconcile depends on Tickets and Auth; new service buys no
resilience.
        Reconcile has the same scaling curve as the rest of the app.
        Modular monolith is the cheaper, more reversible choice.
**Accepted cost:** Future service-extraction will require coordinated
refactoring,
        especially of the Tickets boundary.
**Revisit trigger:** Reconcile traffic exceeds 5× rest-of-app, OR a
second team
        begins contributing reconcile features.

### Trade-off 2 — Sync vs async write to audit
**Tension:** Latency vs durability.
**Option A:** Synchronous write to audit event store before responding
to user.
**Option B:** Async publish to Kafka audit topic; consumer writes to
store.
**Choice:** Option B.
**Why:** Synchronous would add 30–80ms to the reconcile latency budget,
        breaking the p95 ≤ 1000ms SLO. Async with Kafka durability is
sufficient
        for SOC 2 audit requirements. The DLQ on the consumer handles
rare failures.
**Accepted cost:** Audit events may lag 1–10 seconds behind user
action; a
        regulator querying within seconds of an event may see stale
state.
**Revisit trigger:** A regulator or customer requires synchronous audit
visibility
        within < 1 second.
```

What "good" looks like

- Trade-offs span **at least 3 categories**.
- Each names a **real alternative**, not a strawman.

- The "accepted cost" is **concrete** — not "complexity." (What complexity? What does it cost in maintenance, debugging, on-call?)
- The "revisit trigger" is a **metric or organizational change**, not "later."

> Trade-offs that align suspiciously well with personal preference are a smell. The reasoning should come from the system, not from "I like X better." Run a consistency check: do any of your five trade-offs contradict each other? If so, that's a finding — surface it.